User Manual: Hollow Fiber Membrane Solvent Extraction Flux and Mass Transfer Analysis

Rachel Dalke, Thanh-Tin Nguyen, and Lauren Valentino

October 2026

Introduction

Overview

This software provides a Python implementation of the mechanistic mass transfer calculations and model described by Nguyen et al. (2026) for hollow fiber membrane solvent extraction of isobutanol. The software calculates overall mass transfer coefficients (K_{ov}) and associated shell-side, lumen-side, and membrane mass transfer coefficients and resistances from experimental or model input data. The package includes two Excel macro-enabled templates that provide user interfaces for experimental K_{ov} calculation and model-predicted K_{ov} estimation, as well as a Python-based modeling workflow for model building, parameter fitting, model evaluation, and diagnostics. The Python implementation includes modules for core mass transfer calculations, parameter optimization, and evaluation of model predictions against experimental data. An example Jupyter notebook demonstrates application of the model to selected cases from the associated publication.

Please reference the following publication and associated supporting information for equations and experimental design details.

> Thanh-Tin Nguyen, Sunil Shah, Jesse McVay, Meltem Urgun-Demirtas, and Lauren Valentino. (2026). Mechanistic mass transfer in hollow fiber membrane solvent extraction for bio-based isobutanol. *Journal of Membrane Science.* Volume **752**;125544. https://doi.org/10.1016/j.memsci.2026.125544.

This software is intended for researchers, students, technicians, and engineers working in membrane separations, mass transfer, solvent extraction, and related chemical engineering applications. It is particularly applicable to users studying hollow fiber membrane solvent extraction who need to calculate experimental overall mass transfer coefficients, predict mass transfer performance, or develop and evaluate mechanistic mass transfer models. The software is also suitable for users with a basic understanding of Python and Microsoft Excel who wish to analyze experimental data and apply the provided mass transfer model to different operating conditions.

Dependencies

- Python, Microsoft Excel
- A Python environment that supports .ipynb notebooks (e.g., Jupyter Notebook, JupyterLab, Spyder)
- NumPy, Pandas, SciPy, xlwings
- `constants.xlsx`: spreadsheet (provided) containing physical and model constants.

Download instructions

This software requires installation of Python and several associated libraries. If these are not already installed on your local machine, please visit the associated download pages in the Appendix for installation instructions.

Using templates

This software is designed to be compatible with both Windows and Mac devices. Please follow the instructions for your device.

Windows users

1. Download the desired Excel template (.xslm) and associated Python (.py) file.
2. Move these files to a designated working folder, e.g., Desktop.
3. Click on the Excel file to open. Enable macros if needed by following the instructions below:
    1. Close the Excel file completely.
    2. Open File Explorer and find the downloaded file.
    3. Right-click the Excel file and select **Properties**.
    4. On the **General** tab, look at the bottom **Security** section.
    5. Check the box for **Unblock**.
    6. Click **Apply**, then **OK**.
    7. Reopen the file and click **Enable Content** on the yellow security warning bar.
4. Configure file paths in the VBA Editor following the steps below. *These are user-specific and will need to be updated in both templates prior to use.*
    1. In the macro-enabled workbook, right-click the sheet tab at the bottom of your screen and select **View Code** to open the VBA Editor. If the main window appears blank, locate the Project Explorer on the left side of the VBA Editor. If it is not visible, press Ctrl + R on your keyboard to display it.
    2. In the Project Explorer, expand Modules and double-click Module1 to view the code.
    3. Locate the file path variables near the beginning of the code.
    4. Update the `scriptPath` and `pythonExe` variables to their location path on your machine. For example:

        `scriptPath = C:\Users\YourName\Downloads\MSE_Python\templates\kov_exp.py` (Python script file)

        `pythonExe = C:\Users\YourName\.conda\envs\my_env\python.exe` (Python installation)
    5. Save the changes and close the Editor.
5. Run the macro by clicking the `Run kov_exp` or `Run kov_pred` button. Upon completion, an Excel pop-up message will display **Python Finished**.

    On Windows devices, the macro will insert the plotted image directly into Excel. The associated image file can be accessed from the working directory.

Mac users

1. Download the desired Excel template (.xslm) and associated Python (.py) file.
2. Move these files to a designated working folder, e.g., Desktop.
3. Click on the Excel file to open. Excel will pop up a direct dialog box. Click **Enable Macros**.
4. Open Terminal on your Mac. It can be found under Applications > Utilities.
5. In Terminal, navigate to the folder where you placed the files by running:

    ```
    cd /path/to/folder
    ```

    You will need three components, in the following order, to run the program:

    - **Python executible**: the path to your Python installation
    - **Python script**: the path to the Python script file (.py)
    - **Excel workbook**: the path to the associated Excel macro template (.xlsm)
6. Inside the folder where you placed the files, type the following line in Terminal and hit **Enter**.

- Example: `python kov_exp.py kov_exp.xlsm`

NOTE: If you are not in the same directory as the downloaded files, you will need to provide the full path name.

Example of command using full file paths: `path/to/python/installation path/to/python/file path/to/macro-enabled/spreadsheet`

7. Terminal will display **DONE** when finished.

   On Mac devices, the the macro will not insert the plotted image directly into Excel. The associated image file must be accessed from the working directory.

Helpful tips

- To get the file paths on Mac, right-click on the file and select **Get Info**. At the top of the info page, under **General**, find **Where**. Right click in this region and select **"Copy as Pathname"**.
- The exact steps for enabling macros may vary depending on your computer's security settings, Excel version, or organizational policies.

  If macros are blocked by your organization's security policy, contact your IT administrator.

Repository Structure

```
MSE_Python/
├── templates/
│   ├── kov_exp.xlsm
│   ├── kov_exp.py
│   ├── kov_pred.xlsm
│   └── kov_pred.py
├── src/
│   ├── model.py
│   ├── fitting.py
│   ├── conversions.py
│   └── evaluation.py
├── example/
│   ├── example_models.ipynb
│   └── exp_data.csv
├── constants.xlsx
└── program_manual.pdf
```

Part I: Excel Templates

The software includes two Excel-based templates for calculating overall mass-transfer coefficient K_{ov}. Template 1 uses experimental concentration data to determine an experimental K_{ov}, while Template 2 uses model parameters and system operating conditions to predict K_{ov}.

Template 1: Experimental K_{ov} Calculation

Python file path: `MSE_Python/templates/kov_exp.py`

Excel spreadsheet file path: `MSE_Python/templates/kov_exp.xlsm`

Description

This template provides an interface for calculating an experimental overall mass-transfer coefficient (K_{ov}) from time-dependent concentration data. When the program is run, the Python script reads the experimental data and system parameters from Excel, fits the concentration-time data, calculates the corresponding mass-transfer flux and driving force, and determines the experimental K_{ov}.

Inputs

The orange cells mark where experimental data and system parameters (e.g., volumes, densities) are needed for subsequent calculations.

Variable	Description (units)
`time_hr`	Experimental time (h)
`conc_bulk_aq_gm3`	Bulk aqueous-phase concentration (g/m³)
`conc_bulk_org_gm3`	Bulk organic-phase concentration (g/m³)
`system_vol_mL`	System volume (mL)
`m`	Partition coefficient
`active_area_cm2`	Active membrane area (cm²)
`rho_organic`	Organic-phase density (kg/m³)
`rho_aqueous`	Aqueous-phase density (kg/m³)

Outputs

- `Kov_exp`: Experimentally determined, organic-phase-based overall mass-transfer coefficient ($K_{ov,exp,o}$) in m/s.
- b, c, and d: Parameters associated with the fitted exponential concentration-time relationship.
- A plot showing the experimental organic-phase concentration data and the fitted concentration-time model, saved as `fit_plot.png` in the working directory.
- Additional metrics as shown in the table below.

Variable	Description (units)
`driving_force_gm3`	Concentration driving force between the aqueous and organic phases (g/m³)
`conc_bulk_org_gg`	Bulk organic-phase concentration (g/g)
`conc_bulk_aq_gg`	Bulk aqueous-phase concentration (g/g)
`system_mass_g`	Total system mass (g)
`C_model_gg`	Model-predicted organic-phase concentration (g/g)
`dCdt_gg_per_h`	Rate of change of organic-phase concentration (g/g/h)
`J_g_m2_h`	Mass-transfer flux of the component into the organic phase (g/m²/h)

Error Handling

If an error occurs during execution, the script writes an error report to `kov_error.txt` in the working directory. The report contains the error message and Python traceback to assist with troubleshooting.

Template 2: Predicted K_{ov} Estimation

Python file path: `MSE_Python/templates/kov_pred.py`

Excel spreadsheet file path: `MSE_Python/templates/kov_pred.xlsm`

Description

This template provides an interface for calculating a predicted K_{ov} based on the model developed and refined in Nguyen et al., 2026. When the program is run, the Python script reads the system and model parameters from Excel, performs a series of calculations, and returns all determined values.

Inputs

The template contains designated cells for membrane geometry, experimental conditions, and experiment-specific parameters (e.g., density and viscosity of fluids). In the included example, the experiment design involved changes to the number of fibers in the module, as well as the aqueous phase and organic phase flow rates. Experimental constants are in light blue cells, adjusted variables are in dark blue cells, and model parameter inputs are in orange cells.

- A, B, α, and β: model parameters obtained from the modeling functions below.

Outputs

Calculated outputs are in green cells. Key outputs include:

- `ks`, `kl`, and `km`: shell-side, lumen-side, and membrane mass transfer coefficients (k_s, k_l, and k_m), respectively.
- `Kov`: overall mass transfer coefficient K_{ov}.
- `Rs`, `Rl`, `Rm`: shell-side, lumen-side, and membrane mass transfer resistances (R_s, R_m, and R_l), respectively
- `Rov`: overall mass transfer resistance R_{ov}.

Error Handling

If an error occurs during execution, the script writes an error report to `kov_mod_error.txt` in the working directory. The report contains the error message and Python traceback to assist with troubleshooting.

Part II: MSE Model Example

Launching the notebook

The example notebook is provided as an `.ipynb` file and can be run using Jupyter Notebook, JupyterLab, Spyder, or another compatible Python environment. The software can be run using Anaconda, Miniconda, or another Python distribution with Conda support. Before running the notebook, ensure that the required dependencies are installed in the Python environment used to run the notebook.

Option 1. Using Anaconda Navigator:

1. Open Anaconda Navigator.
2. Select the Python environment containing the required dependencies.
3. Launch Jupyter Notebook or JupyterLab.
4. Navigate to the directory containing the example notebook.
5. Open the .ipynb file.
6. Run the notebook cells sequentially from the beginning.

Option 2. Using the Anaconda Prompt (Windows) or Terminal (Mac)

1. `conda activate <environment_name>`
2. `cd <path_to_notebook>`
3. `jupyter notebook` or `jupyter lab` (or call another notebook environment)
4. Open the .ipynb file.
5. Run the notebook cells sequentially from the beginning.

Description

The accompanying coding notebook shows the MSE modeling workflow in practice. Selected experimental cases from Nguyen et al. (2026) are used to demonstrate the model-building, parameter-fitting, and evaluation functions. The workflow consists of preparing the experimental and system data, fitting model parameters to selected experimental datasets, and evaluating the resulting model against experimental data.

Two primary sources of information are required:

1. **Experimental data** (`exp_data.csv`) — contains the experimental conditions and experimentally determined K_{ov} values used for regression and evaluation. In this example, Experiments 1 and 2 reflect experiments investigating membrane packing fraction and shell-side flow dynamics, respectively.
2. **Model constants** (`constants.xlsx`) — contains physical properties, membrane dimensions, and other parameters that remain fixed across experiments. Examples include the density of solvents used, fixed aspects of membrane geometry, etc. These can be ajusted prior to running the model by accessing the `constants.xlsx` file and adjusting the associated cell references.

   Note: If you update the units of the input parameters, be sure to change the associated conversion in `conversions.py`!

The initial portion of the notebook covers directory setup, necessary imports, unit conversions, and specification of the experimental datasets. Experimental datasets can be used either for **regression**, in which model parameters are estimated, or for **evaluation**, in which previously fitted parameters are used to predict experimental results without further parameter fitting. This distinction allows the model to be tested not only for how well it describes the data used during parameterization, but also for how well it predicts experiments conducted under different operating conditions.

The model contains four parameters, A, B, α, and β. Different parameter-fitting cases can be used to investigate how much flexibility is required to reproduce the experimental data. In each case, some parameters are allowed to vary during regression while others are held fixed. Comparing these cases allows the user to determine whether additional adjustable parameters improve the model's ability to describe the experimental data. The example also includes a determination of the difference between the reported R^2 values in Nguyen et al. (2026) and the Python implementation.

The example notebook progresses through three stages:

1. **Packing-fraction model.** Experimental data are used to evaluate which combinations of model parameters best describe changes in packing fraction.
2. **Flow-rate evaluation.** The fitted models are applied to experiments performed at different aqueous-phase flow rates. The parameters are not re-fitted during this step, allowing the predictive capability of the model to be assessed.
3. **Combined model.** Because the packing-fraction-only model does not adequately describe the flow-rate experiments, both datasets are used to develop a model that accounts for both effects. Previously fitted parameters are used as starting values for the new regression.

   Heplful tip: If an error occurs or the notebook gets into an unexpected state, restart the kernel and clear the existing outputs before running the notebook again. This can be done using Kernel → Restart Kernel and Clear Outputs (menu names may vary by environment).

Part III. Mass Transfer Model Function Library

This package contains 3 modules for reproducing the the resistance-in-series model described in Nguyen et al. 2026. The modules enable model building, fitting, and evaluation.

Modules

- `src/model.py` contains core mass-transfer calculations.
- `src/fitting.py` is used for parameter fitting and model configuration.
- `src/evaluation.py` is used for model evaluation and diagnostics.
- `src/conversions.py` contains a dictionary of mathematical operations for converting typical parameters into SI units.

Workflow

1. Define physical constants.
2. Load experimental data.
3. Specify model parameters to fit.
4. Call `fit_model()`.
5. Evaluate fitted parameters using `evaluate_model()`.

The following subsections describe the Python modules and their associated functions.

Mass-Transfer Model: `model.py`

The `model.py` module contains the core mass-transfer calculations used to predict the overall mass-transfer coefficient K_{ov}. It calculates the mass-transfer coefficients and resistances on the shell, lumen, and membrane sides of the system using the specified model parameters, physical constants, and experimental operating conditions. The module provides functions for obtaining either the overall K_{ov} prediction or the complete set of intermediate model calculations.

`calculate_kvals()`

Inputs

- `params`: Dictionary of parameters for the shell-side Sherwood number correlation: A, B, α, and β.
- `constants`: Dictionary containing the physical properties, diffusion coefficients, membrane properties, and module geometry required by the model. These include fluid viscosities and densities, diffusion coefficients, membrane and fiber dimensions, module length, membrane porosity, tortuosity, and membrane thickness.
- `df`: Pandas DataFrame containing the operating-condition information required for the calculations. Required columns include:

Variable	Description (units)
`Q_a_m3s`	Aqueous-phase flow rate (m^3/s)
`Q_o_m3s`	Organic-phase flow rate (m^3/s)
`NF`	Number of fibers in the module
`m`	Partition coefficient

Outputs

Returns the complete dictionary of model calculations produced by `calculate_kvals()`, including K_{ov}, the individual mass-transfer coefficients, dimensionless quantities, and mass-transfer resis-

tances. This function provides intermediate quantities needed for detailed model diagnostics and analysis.

Description

Performs the complete mass-transfer calculation for the specified operating conditions and model parameters. The function first calculates the shell-side geometry and flow quantities, followed by shell-side Reynolds, Schmidt, and Sherwood numbers and the corresponding mass-transfer coefficient. It then calculates the corresponding lumen-side quantities and mass-transfer coefficient using a Graetz-number-dependent Sherwood correlation. The membrane mass-transfer coefficient is calculated from the membrane properties. The shell-side, lumen-side, and membrane resistances R_s, R_l, and R_m are combined to calculate the overall mass-transfer coefficient K_{ov}. The function returns both K_{ov} and the intermediate quantities used in its calculation.

`predict_kov()`

Inputs

Requires same inputs as defined under `calculate_kvals()`.

Output

- `Kov`: Array of predicted overall mass-transfer coefficients (K_{ov}) corresponding to the rows of the input DataFrame.

Description

Calculates the predicted overall mass-transfer coefficient K_{ov} using `calculate_kvals()` and returns only the resulting K_{ov} values. This function is complementary to `calculate_kvals` and provides a simplified interface for applications where only the overall mass-transfer coefficient is required, such as model evaluation and parameter fitting.

Parameter Fitting: `fitting.py`

The `fitting.py` module contains functions for organizing datasets, defining fixed and fitted model parameters, calculating the fitting objective function, and optimizing the model parameters. The fitting procedure uses the model predictions from model.py and minimizes a normalized sum of squared errors between predicted and experimental K_{ov} values.

`get_dataset_split()`

Inputs

- `datasets`: Dictionary containing the available datasets, indexed by dataset name.
- `experiment`: Dictionary specifying which datasets are used for regression and which are used for evaluation. The dictionary may contain regression and evaluation entries containing lists of dataset names.

Outputs

- `regression_sets`: List of datasets designated for parameter fitting.
- `evaluation_sets`: List of datasets designated for model evaluation.

Description

Separates the available datasets into regression and evaluation groups according to the dataset names specified in the experiment configuration. This allows datasets used to fit the model parameters to be distinguished from datasets reserved for independent evaluation.

`compile_params()`

Inputs

- `fixed`: Dictionary specifying the values of the model parameters. Parameters assigned a numerical value are held fixed during fitting, while parameters assigned None are treated as free parameters.
- `param_names`: List containing the names of the model parameters to be considered.

Outputs

- `unpack`: Function that converts an array of free parameter values into a complete parameter dictionary containing both fitted and fixed parameter values.
- `n_free`: Number of model parameters being fitted.

Description

Determines which model parameters are fixed and which are free to vary during optimization. It creates an internal function that reconstructs the complete parameter dictionary from the values of the free parameters returned by the optimization routine.

`least_squares()`

Inputs

- `free_params`: Current values of the free model parameters being evaluated during optimization.
- `datasets`: Dataset or collection of datasets containing experimental K_{ov} values in the `Kov_exp` column.
- `constants`: Physical and model constants required to calculate the predicted K_{ov} values.
- `unpack`: Function used to reconstruct the complete model parameter dictionary from the free parameters.

Output

- `total`: Normalized sum of squared errors between predicted and experimental K_{ov} values across the supplied datasets.

Description

Calculates the objective function minimized during parameter fitting. The function reconstructs the complete parameter set, calculates predicted K_{ov} values using `predict_kov()`, and compares the predictions with the experimental values. The squared errors are normalized by the mean experimental K_{ov} value for each dataset and summed across all datasets.

`fit_model()`

Inputs

- `regression_sets`: Collection of datasets used for fitting the model parameters. Each dataset must contain experimental K_{ov} values in the `Kov_exp` column.
- `fixed`: Dictionary specifying which model parameters are fixed and their values. Parameters set to None are fitted.
- `constants`: Physical and model constants required by the model.
- `x0`: Optional initial values for the free model parameters. If not provided, an initial value of 1 is used for each free parameter.

Outputs

- `params`: Dictionary containing the fitted values of all model parameters, including both optimized and fixed parameters.
- `result`: Optimization result returned by SciPy's `minimize()` function. This contains information about the optimization process, including the optimized parameter vector and convergence information.

Description

Fits the model parameters using the regression datasets. The function defines the model parameters (A, B, α, and β), determines which parameters are free to vary, establishes their initial values, and uses `scipy.optimize.minimize()` to minimize the normalized sum of squared errors calculated by `least_squares()`. The resulting optimized parameter values are combined with any fixed parameter values and returned together with the full optimization result.

`modify_case()`

Inputs

- `base_params`: Dictionary containing a baseline set of model parameters.
- `changes`: Keyword arguments specifying parameter values to be changed from the baseline parameter set.

Output

- `new_case`: Copy of the baseline parameter dictionary with the specified parameter values replaced by the supplied values.

Description

Creates a modified parameter set from an existing set of model parameters without changing the original dictionary. This function can be used to generate alternative parameter cases for subsequent model calculations or analysis.

Model Evaluation: `evaluation.py`

The `evaluation.py` module contains functions for assessing model predictions against experimental K_{ov} values. It provides both a general model evaluation function that calculates an overall coefficient of determination (R^2) and a diagnostic function that generates detailed error metrics and tabulated results for individual datasets and for the datasets collectively.

`evaluate_model()`

Inputs

- `params`: Set of model parameters used to calculate predicted K_{ov} values.
- `evaluation_sets`: Collection of datasets used to evaluate the model. Each dataset must contain experimental K_{ov} values in a column named `Kov_exp`.
- `constants`: Physical or model constants required by the K_{ov} prediction calculation.

Outputs

- `r2`: Overall coefficient of determination (R^2) calculated across all evaluation datasets.
- `all_results`: List of evaluation datasets containing the original data and an additional `Kov_pred` column containing the model-predicted K_{ov} values.

Description

Calculates model-predicted K_{ov} values for each dataset using `predict_kov()` and compares them with the corresponding experimental values. For each dataset, the function calculates the residual and total sums of squares and uses these quantities to calculate R^2. The residual and total sums of squares are then combined across all datasets to obtain an overall R^2 value. The function returns the overall R^2 and the datasets containing the model predictions.

`diagnose_model()`

Inputs

- `params`: Set of model parameters used to calculate the model predictions and associated mass-transfer quantities.
- `datasets`: Collection of datasets used for detailed model diagnostics. Each dataset must contain experimental K_{ov} values in a column named `Kov_exp`.
- `constants`: Physical or model constants required by the model prediction calculation. See included spreadsheet `constants.xlsx`.

Outputs

- `datasets`: List of diagnostic results for each input dataset. Each result contains the dataset-specific R^2 value, normalized error, and a table containing model outputs, predicted and experimental K_{ov} values, and error metrics.
- `combined_r2`: Overall coefficient of determination (R^2) calculated across the processed datasets.
- `combined_error`: Combined normalized squared error accumulated across the processed datasets.

Description

Generates detailed diagnostic information by calculating model predictions and comparing them with experimental K_{ov} values. For each dataset, the function calculates the absolute relative error, squared prediction error (SSE), total squared deviation from the experimental mean (SST), and normalized squared error. A dataset-specific R^2 and normalized error are calculated from these quantities.

The function also creates an output table containing the calculated mass-transfer coefficients (k_s, k_m, and k_l), predicted and experimental K_{ov} values, reciprocal predicted K_{ov} values, scaled K_{ov} values, and the calculated error metrics. Diagnostic results are stored for each dataset, while the corresponding error and sum-of-squares quantities are accumulated to calculate combined metrics across the datasets.

Example

Notebook file path: `MSE_Python/example/example_models.ipynb`

Example data file path: `MSE_Python/example/exp_data.csv`

Additional required components

`conversions.py`

This module contains only a dictionary of unit conversions. It is coupled to the `constants.xslx` spreadsheet and is within the Jupyter notebook example to convert the units typically observed in hollow fiber membrane experiments (cP, g/cm³, cm²/s, etc.) into SI units prior to model fitting.

`exp_data.csv`

This comma-separated values file contains experimental data from the Nguyen et al. (2026) study. More information on this data is provided in the example Jupyter notebook file.

Description

Example application demonstrating use of the model. This is an interactive Jupyter notebook file following the setup and implementation of selected cases in the Nguyen et al. study. The determination coefficient analysis of $k_{s,\varphi}$ using different parameter regression (Cases I-V) and the subsequent re-analysis for Cases III-V are provided (see Nguyen et al., Figures S5, S6), showing the reduction in R^2 observed in the original study. To demonstrate parameter optimization, an implementation of Case VIII is shown, where the two parameters A and β were simultaneously optimized.

Appendix

Model Implementation and Results

- Minor discrepancies were observed in in k_l calculation due to non-rounding in the calculation of Sh_l. Propagation of rounded values in the original Excel documentation may explain some minor numerical differences.
- The target membrane area shown in the modeling example notebook reflects the intended area during module design (Nguyen et al., 2026, Table 2) and is provided for reference. The membrane area supplied in Template 1 is determined after fabrication and is the area used in the determination of the experimental K_{ov}.
- The Python implementation utilizes `scipy.optimize.minimize()`, which operates differently than the GRG Nonlinear Solver in Microsoft Excel, which may explain some of the observed numerical deviation.

Third-Party Code Information

The software was developed and tested using the following software versions: Python packages NumPy v2.4.4, Pandas v3.0.2, SciPy v1.17.1, xlwings v0.35.1, and Matplotlib v3.10.9, with Jupyter Notebook v7.5.5 used for the interactive modeling workflow. Links to all third-party code embedded in or accessed by this package is shown in the table below.

Library	Web Page	License
Numpy	https://numpy.org/	https://github.com/numpy/numpy/blob/main/LICENSE.txt
Pandas	https://pandas.pydata.org/	https://github.com/pandas-dev/pandas/blob/main/LICENSE
SciPy	https://scipy.org/	https://github.com/scipy/scipy/blob/main/LICENSE.txt
xlwings	https://docs.xlwings.org/en/latest/installation.html#installation	https://docs.xlwings.org/en/latest/license.html
Matplotlib	https://matplotlib.org/	https://matplotlib.org/stable/project/license.html
Jupyter Notebook	https://jupyter.org/install	https://jupyter.org/governance/projectlicense/